



Technische Hochschule
Ingolstadt

Fakultät
Maschinenbau

Autonomes Fliegen

WS 2023

***Prof. Dr. Gerhard Elsbacher,
and Dr. techn. Babak Salamat***

26.10.2023



- Kapitel 1: Übersicht über Unmanned Aerial Systems (UAVs) (1 W)
- Kapitel 2: Wesentliche Komponenten/Subsysteme eines UAV (1 W)
- Kapitel 3: Modellierung und Simulation von UAVs (1 W)
- Kapitel 4: Flight State Controller (PD Cascade) (1W)
- **Kapitel 5: Pfadplanung (4 W)**
- Kapitel 6: Perception und Sensing (1 W)
- Kapitel 7: Navigation (2 W)
- Kapitel 8: Multi-Vehicle Cooperation and Coordination (1 W)

Trajectory Planning

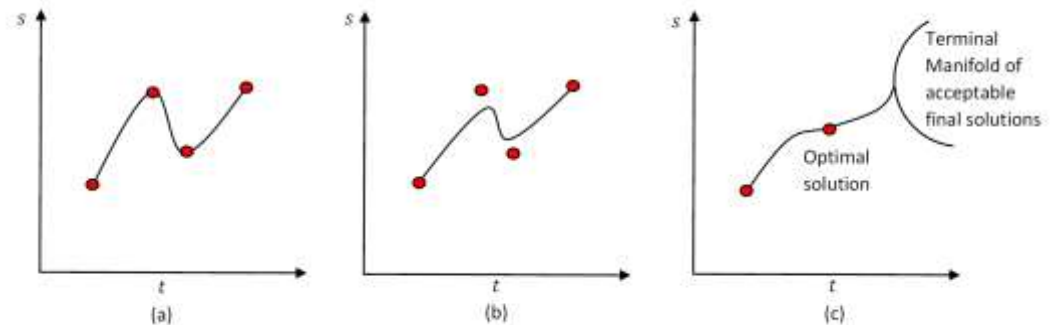
Some suggested references:

- C. Melchiorri, *Traiettorie per azionamenti elettrici*, Progetto Leonardo, Esculapio Ed., Bologna, Feb. 2000;
- G. Canini, C. Fantuzzi, *Controllo del moto per macchine automatiche*, Pitagora Ed., Bo, 2003;
- G. Legnani, M. Tiboni, R. Adamini, *Meccanica degli azionamenti: Vol. 1 - Azionamenti elettrici*, Progetto Leonardo, Esculapio Ed., Bologna, Feb. 2002.
- L. Biagiotti, C. Melchiorri, *Trajectory Planning for Automatic Machines and Robots*, Springer, 2008.



Springer, 2008

- Introduction
- Joint-space trajectories
- Polynomial trajectories
- Trapezoidal trajectories
- Spline trajectories



Static interpolation (a), approximation (b), and dynamic interpolation (c)

Kinematics: geometrical relationships in terms of position/velocity between the joint- and work-space.

Dynamics: relationships between the torques applied to the joints and the consequent movements of the links.

Control: computation of the control actions (joint torques) necessary to execute a desired motion.

Trajectory planning: planning of the *desired* movements of the manipulator.

Usually, the user is requested to define some points and general features of the trajectory (e.g. initial/final points, duration, maximum velocity, etc.), and the real computation of the trajectory is demanded to the control system.

The planning modalities for trajectories may be quite different:

- point-to-point
- with pre-defined path

Or:

- in the joint space;
- in the work space, either defining some points of interest (initial and final points, *via points*) or the whole geometric path $\mathbf{x} = \mathbf{x}(t)$.

For planning a desired trajectory, it is necessary to specify two aspects:

- *geometric path*
- *motion law*

with constraints on the continuity (smoothness) of the trajectory and on its time-derivatives up to a given degree.

Trajectory Planning

Motion law

Examples of geometric paths: (in the work space) linear, circular or parabolic segments or, more in general, tracts of analytical functions.

In the joint space, geometric paths are obtained by assigning initial/final (and, in case, also intermediate) values for the joint variables, along with the desired motion law.

Concerning the **motion law**, it is necessary to specify continuous functions up to a given order of derivation (often at least first and second order, i.e. velocity and acceleration).

Usually, polynomial functions with a proper degree n are employed:

$$s(t) = a_0 + a_1 t + a_2 t^2 + \dots + a_n t^n$$

In this manner, a “smooth” interpolation of the points defining the geometric path is achieved.

Input data to an algorithm for trajectory planning are:

- data defining on the path (points),
- geometrical constraints on the path (e.g. obstacles),
- constraints on the mechanical dynamics
- constraints due to the actuation system

Output data is:

- the trajectory in the joint- or work-space, given as a sequence (in time) of the acceleration, velocity and position values:

$$a(kT), \quad v(kT), \quad p(kT) \quad k = 0, \dots, N$$

being T a proper time interval defining the instants in which the trajectory is computed (and in case converted in the joint space) and sent to each actuator.

Usually, the user has to specify only a minimum amount of information about the trajectory, such as initial and final points, duration of the motion, maximum velocity, and so on.

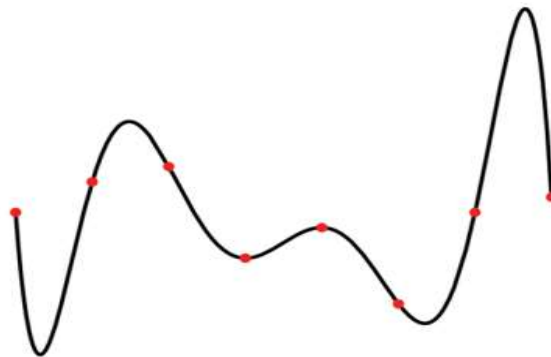
- *Work-space trajectories* allow to consider directly possible constraints on the path (obstacles, path geometry, ...), that are more difficult to take into consideration in the joint space (because of the non linear kinematics)
- *Joint space trajectories* are computationally simpler and allow to consider problems generated by singular configurations, actuation redundancy, velocity/acceleration constraints.

Trajectories are specified by defining some characteristic points:

- directly assigned by some specifications
- assigned by defining desired configurations $\mathbf{x}$ in the work-space, which are then converted in the joint space using the inverse kinematic model.

The algorithm that computes a function $\mathbf{q}(t)$ *interpolating* the given points is characterized by the following features:

- trajectories must be computationally efficient
- the position and velocity profiles (at least) must be continuous functions of time
- undesired effects (such as non regular curvatures) must be minimized or completely avoided.



Polynomial Trajectories

Joint-space trajectories

In the most simple cases, (a segment of) a trajectory is specified by assigning initial and final conditions on: time (duration), position, velocity, acceleration, Then, the problem is to determine a function

$$q = q(t) \quad \text{or} \quad q = q(\sigma), \quad \sigma = \sigma(t)$$

so that those conditions are satisfied.

This is a **boundary condition** problem, that can be easily solved by considering polynomial functions such as:

$$q(t) = a_0 + a_1 t + a_2 t^2 + \dots + a_n t^n$$

The degree n (3, 5, ...) of the polynomial depends on the number of boundary conditions that must be verified and on the desired “smoothness” of the trajectory.

In general, besides the initial and final values, other constraints could be specified on the values of some time-derivatives (velocity, acceleration, jerk, ...) in generic instants $t_j \in [t_i, t_f]$.

In other terms, one could be interested in defining a polynomial function q whose k -th derivative has a specified value $q^k(t_j)$ at a given instant t_j .

Mathematically, these conditions may be expressed as:

$$k!a_k + (k+1)!a_{k+1}t_j + \dots + \frac{n!}{(n-k)!}a_nt_j^{n-k} = q^k(t_j)$$

or, in matrix form:

$$\mathbf{M} \mathbf{a} = \mathbf{b}$$

where:

- $\mathbf{M}$ is a known $(n+1) \times (n+1)$ matrix,
- $\mathbf{b}$ is the vector with the $n+1$ constraints on the trajectory (known data),
- $\mathbf{a} = [a_0, a_1, \dots, a_n]^T$ contains the unknown parameters to be computed.

Third-order Polynomial Trajectories

Given an initial and a final instant t_i, t_f , a (segment of a) trajectory may be specified by assigning initial and final conditions:

- initial position and velocity $q_i, \dot{q}_i$;
- final position and velocity $q_f, \dot{q}_f$

There are **four boundary conditions**, and therefore a polynomial of (at least) degree 3 must be considered

$$q(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 \quad (1)$$

where the **four parameters** a_0, a_1, a_2, a_3 must be defined so that the boundary conditions are satisfied. From the boundary conditions, it follows that

$$\begin{aligned} q(t_i) &= a_0 + a_1 t_i + a_2 t_i^2 + a_3 t_i^3 &= q_i \\ \dot{q}(t_i) &= a_1 + 2a_2 t_i + 3a_3 t_i^2 &= \dot{q}_i \\ q(t_f) &= a_0 + a_1 t_f + a_2 t_f^2 + a_3 t_f^3 &= q_f \\ \dot{q}(t_f) &= a_1 + 2a_2 t_f + 3a_3 t_f^2 &= \dot{q}_f \end{aligned} \quad (2)$$

In order to solve these equations, let us assume for the moment that $t_i = 0$.
Therefore:

$$a_0 = q_i \quad (3)$$

$$a_1 = \dot{q}_i \quad (4)$$

$$a_2 = \frac{-3(q_i - q_f) - (2\dot{q}_i + \dot{q}_f)t_f}{t_f^2} \quad (5)$$

$$a_3 = \frac{2(q_i - q_f) + (\dot{q}_i + \dot{q}_f)t_f}{t_f^3} \quad (6)$$